8. Packages, Modules, Visibility, and go.mod

08. Packages, Modules, Visibility, and go.mod

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `08-packages-modules`

Learning objectives

- Create modules with go.mod
- Export identifiers with Capital letters
- Import packages and avoid import cycles
- Organize code into packages with a single purpose

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Modules

A **module** is a versioned collection of packages (`go.mod`).

```
go mod init github.com/you/project
go get example.com/some/dep@v1.2.3
go mod tidy
```

Packages

Every Go file starts with `package name`. Directory ≈ package (usually). `package main` with `func main()` builds an executable.

Visibility

- `Name` (capital) → exported, visible outside the package
- `name` (lowercase) → package-private

There is no `private` keyword. This simplicity scales surprisingly well.

Import paths

```
import (
    "fmt"
    "github.com/you/project/internal/auth"
)
```

`internal/` directories are importable only by the parent tree — enforce encapsulation.

Common mistakes (especially from Python instincts)

- Creating many tiny packages too early (start coarser)
- Import cycles from poor boundaries

- Putting everything in `package main` forever

Exercises

Work in `exercises/08-packages-modules/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/08-packages-modules/` until you have an honest failing attempt.

Self-check

1. How do you export a function?
2. What does `go mod tidy` do?
3. Why does `internal/` exist?

Further reading

- https://go.dev/doc/modules/gomod-ref
- https://go.dev/doc/code